



A Cookiecutter Template for Ab-initio Complex Modelling

Generalizing the ABCfold pipeline: architecture, templating, and reporting

Etienne Reboul

Groupe Gibbérellines et adaptation à l'environnement
Institut de Biologie Moléculaire des Plantes (IBMP), Strasbourg

September 21, 2026

Outline

1. Introduction
2. The three-stage pipeline
3. Turning it into a template
4. One config, three schedulers
5. Reports instead of notebooks
6. Does it actually work?
7. Next steps

Introduction

Where this comes from

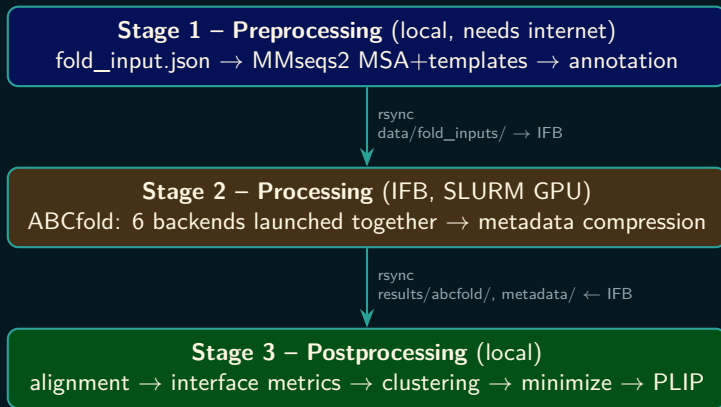
- Three one-off pipelines already run the **same** 3-stage protocol (preprocess → process → postprocess):
 - NPF / ABCfold (gibberellin importers – this lab's prior talk)
 - DRB2/DCL4/dsRNA
 - AGO1/FBW2/miR
 - Each new dataset meant **copy the pipeline, hand-edit the scripts** – system names, chain ids, and domain ranges hardcoded throughout
- ⇒ Same skeleton, three separate codebases to maintain and keep in sync

Goals of the generalization

1. One reusable template: **generate-and-run**, not copy-and-edit
2. Adding a new dataset = editing **only** config.yaml's systems: list + one configs/<system>.yaml – **no script edits**
3. Portable across HPC schedulers: SLURM, PBS, HTCondor
4. **Report-first**: every stage ships a self-contained HTML report bundle, replacing ad hoc notebook visualization

The three-stage pipeline

Three stages, one diagram



Each stage ends in a self-contained report bundle (reports/<stage>.zip).

Stage 1 – Preprocessing

- **MMseqs2** against the ColabFold webserver: MSA + top-hit template search – offloaded, no local sequence database needed
 - **Domain annotation:** ScanProsite + InterProScan REST
 - **Disorder prediction:** IUPred3 (long/short) + ANCHOR2 + AIUPred
 - **MoRF annotation:** MoRFchibi2 (tools/MC2)
- ⇒ `fold_input.json` (holo & apo forms), with a **human-review gate**:
`annotation_reviewed: false` blocks stage 2 until the curated domains:
block is reviewed

Stage 2 – Processing

- Snakemake, with a **pluggable executor plugin** per HPC scheduler (SLURM/PBS/HTCondor – next section)
- One SLURM GPU job per system: **ABCfold** runs all 6 backends together (`-abcopr`)
- Validated on IFB: job 1840322, 5 backends $\times$ 3 seeds = 15 models, $\sim$ 25 min on an L40S node
- Metadata compression: per-model `ptm/iptm/ ranking_score` $\rightarrow$ `arrays.h5` + `model_metadata.parquet`

The six backends

- AlphaFold3

Abramson *et al.*, *Nature* 2024

- Boltz-2

Passaro *et al.*, *bioRxiv* 2025

- Chai-1

Chai Discovery team, *bioRxiv* 2024

- OpenFold3

OpenFold Consortium, technical report, 2025

- Protenix

ByteDance AI4Science, *bioRxiv* 2025

- RosettaFold3

Baker/DiMaio Lab (IPD), Microsoft Foundry, 2025

All launched via ABCfold, a wrapper that runs multiple backends in one call – pooling backends recovers conformational diversity a single model misses (see prior talk).

Stage 3 – Postprocessing

1. **Kabsch alignment** on each rigid anchor (`anchor_chains`), iteratively refined
2. **Interface metrics** over the full ensemble: ipSAE, iLIS, Pinc
3. **Pose clustering**: HDBSCAN+Optuna/DBCV, or GMM/BIC
4. Top-N/cluster → **ChimeraX minimize** → `fix_pdb` → **PLIP** × passes → aggregate

`plip_passes` generalises what used to be several hardcoded PLIP configurations (protein–protein, RNA–ligand, mixed) into one configurable list per system.

Turning it into a template

The audit

- Before templating: does anything in the pipeline or reporting logic hardcode a system name, chain id, organism, or account *outside* the config layer?
- Config layer = `config.yaml` + `configs/<system>.yaml` + `config.local.yaml`
- Found only a short, enumerable list of non-generic leftovers:
 - 2 standalone scripts hardcoding DRB2/DRB4 disorder ranges – **not referenced by any Snakefile**
 - one `_reference/` directory (old submission script, kept for reference)
 - one legacy config-key back-compat shim, dead code

⇒ **Verdict: go ahead.**

Template mechanics

Repo root is the template: `cookiecutter.json + hooks/` at root; every real pipeline file lives under `{{cookiecutter.project_slug}}/`.

prompt	feeds
<code>first_system_name</code>	<code>config.yaml systems: + configs/<name>.yaml</code>
<code>project_slug</code>	generated directory name
<code>contact_email</code>	<code>config.local.yaml</code> (InterProScan REST contact)
<code>scheduler</code>	<code>config.yaml hpc.scheduler</code> (slurm/pbs/htcondor)
<code>cluster_account</code>	<code>config.yaml hpc.account</code>
<code>chimerax_bin</code>	stage 3 minimization path
<code>backends</code>	subset of the 6 ABCfold backends
<code>setup_first_system_now</code>	triggers interactive FASTA → config flow

Pre/post-gen hooks

- **Pre-gen:** validates `project_slug/` `first_system_name` are filesystem-safe, and that `contact_email` isn't blank or the placeholder – EBI's InterProScan REST service rejects requests without a real contact address
- **Post-gen:**
 - drops sibling-only leftovers (the 2 DRB2 scripts, `_reference/`, empty `report/captions/`)
 - **writes** `config.local.yaml` from `contact_email` – this file had always been hand-created before; it was the explicit trigger for this whole effort
 - optional: interactive FASTA → `configs/<system>.yaml` flow (chain ids, anchor/partner, size class)
 - `git init` + initial commit

Adding a new dataset

Edit **only**: 1 line in config.yaml's systems: list + 1 new configs/<system>.yaml. **No script edits.**

```
sequences:
  - {type: protein, id: A, name: Receptor, sequence: "..."}
  - {type: protein, id: B, name: Partner, sequence: "..."}
anchor_chains: [A]
partner_chains: [B]
annotation_reviewed: false
domains:
  A: [{name: dom1, start: 1, end: 76, kind: domain}]
plip_passes:
  - {name: main, dnareceptor: false, fix_pdb: false, chains: "[['A'], ['B']]"}

```

One config, three schedulers

Why `hpc:`, not `cluster:`

- Named `hpc:` to avoid colliding with the pre-existing `pose-cluster:` key used by stage 3
 - One sub-block per scheduler (`slurm/pbs/htcondor`), each following **that scheduler's own** Snakemake executor plugin's actual documented resource-key conventions – not a made-up common schema
 - `run_abcfold` declares the **union** of all three schedulers' resource keys; a `res_*` function gated on `hpc.scheduler` resolves each one
- ⇒ inactive schedulers' keys resolve to harmless `0/""` – no Jinja/conditional templating of the Snakefile itself needed

Validated vs. untested – be honest

- Only **SLURM** has run a real job – IFB, job 1840322
- pbs/htcondor: wired from each plugin's own README, confirmed to render valid config/resource values for all three scheduler choices (cookiecutter `--no-input` + isolated resolver-function tests) – but **never run against a live pool**
- Known gaps if you try one:
 - PBS: no native GPU resource; account/queue can't vary by `size_class`
 - HTCCondor: assumes a shared filesystem between submit/execute nodes

Reports instead of notebooks

Design goals

- Replace ad hoc notebook visualization with **Snakemake's built-in report**: one self-contained bundle per stage (`reports/<stage>.zip`)
 - **SVG/PDF figures only**, never raster
 - **Dark theme by default** (`report/custom.css`, injected via `--report-stylesheet` – the underlying report app has no native dark mode); `datavzrd` tables get a matching dark skin
- ⇒ **Zip, not bare .html**: browsers block the data: URLs a single-file report would need to embed interactive tables

Stage 1 & 2 reports

Stage 1:

- ColabFold-style MSA coverage/depth plot
- Domain bar chart, segmented by domain/linker
- Superposed disorder line plots (IUPred3 long/short, ANCHOR2, AIUPred)
- datavzrd tables: input-file statistics, template-hit coverage

Stage 2:

- datavzrd table: metadata-compression space saved

Stage 3 report

- RMSD violin plots + rigid-core line plot (which residues anchor the Kabsch alignment)
- Per-backend `datavzrd` failure-rate table + seaborn energy-minimization line plot
- Customisable dimensionality-reduction panels (PCA/t-SNE/UMAP/MDS), colored by backend, rescoring metric, or cluster
- PLIP contact heatmaps: per backend / per cluster / total – domain-aware for protein–protein, nucleic-acid-aware for protein–NA/ligand interfaces

Does it actually work?

End-to-end toy validation

`example_toy` (ubiquitin homodimer) through all 3 stages:

stage	where	result
1 – preprocessing	local	DAG-clean
2 – processing	IFB SLURM GPU	job 1840322, 15 models, ~25 min (L40S)
3 – postprocessing	local (ChimeraX/Docker)	53/53 steps clean

Rough edges found & fixed along the way: a stale `snakemake-module` SLURM executor plugin that hung silently at job submission, and resource-key restrictions in plugin ≥ 2.7 – both are now committed fixes, not open issues.

Current status, honestly

- **Proven:** the SLURM path, the reporting layer, the cookiecutter mechanics themselves (repeated `cookiecutter --no-input` runs, all three scheduler choices)
- **Still open:**
 - PBS/HTCondor unproven against a live pool
 - `report/` vs. `reports/` naming – a real footgun, never resolved
 - `docstring/Usage`: cleanup sweep only partially done

Next steps

Roadmap

- Push this repo to GitHub, so `cookiecutter gh:EtienneReboul/ab_initio_pipeline` works directly
- Get PBS or HTCondor access to actually validate those paths
- Apply the template to the **next real dataset** – the real test of “2 files, no script edits”
- Decide the `report//reports/` naming question

Questions?